Projet Intégration et Déploiement Continus

TP DevOps — Docker, Kubernetes, CI/CD

Rayane Achouchi

24 avril 2026

github.com/RayaneChCh-dev/k3s-tp

Application full stack (Frontend Nginx + Backend Flask + PostgreSQL)
Orchestration locale Docker Compose, déploiement Kubernetes via k3s
sur une VM Azure, automatisé par un pipeline GitHub Actions.

1. Contexte et objectifs

Une entreprise souhaite automatiser le déploiement de son application web afin de gagner du temps et d'éviter les erreurs humaines. L'objectif du TP est de mettre en place une chaîne complète : conteneurisation d'une application, orchestration locale, déploiement Kubernetes sur une machine virtuelle dans le cloud, et pipeline CI/CD qui relie le tout.

L'application retenue est un mini-dashboard composé de trois services distincts qui communiquent entre eux :

- un frontend Nginx servant une page HTML avec dashboard des endpoints ;
- un backend Python (Flask) exposant 5 endpoints REST ;
- une base PostgreSQL 15 initialisée par un script SQL.

Stack technique retenue

```
Frontend         Nginx + HTML + JavaScript vanille
Backend          Python 3.11 / Flask
Base de données  PostgreSQL 15 (image Alpine)
Conteneurisation Docker (multi-stage builds)
Orchestration    Docker Compose v2 (local) / k3s (cloud)
Cloud            Microsoft Azure (VM Ubuntu 22.04)
CI/CD            GitHub Actions
Registry         Docker Hub (saphir10036/*)
```

Architecture cible

```
Navigateur --> VM Azure (74.161.44.42)
                 |
                 +--> k3s cluster
                        namespace: devops-tp-rayane
                          frontend (NodePort 30080)
                             |
                             v
                          backend (ClusterIP :5000)
                             |
                             v
                          db (ClusterIP :5432) -- Secret db-secret
```

2. Conteneurisation avec Docker

2.1 Dockerfile backend (multi-stage)

Le backend utilise un build en deux étapes. La première (builder) installe les dépendances Python avec pip dans /root/.local ; la seconde (runtime) récupère uniquement ces artefacts et la source, ce qui réduit la taille finale de l'image et évite d'embarquer la toolchain de build.

```
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --user --no-cache-dir -r requirements.txt

FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /root/.local /root/.local
COPY app.py ./
ENV PATH=/root/.local/bin:$PATH
EXPOSE 5000
HEALTHCHECK --interval=30s CMD curl -fsS http://localhost:5000/health || exit 1
CMD ["python", "app.py"]
```

2.2 Dockerfile frontend (multi-stage)

Le frontend suit le même principe : un stage builder prépare les assets statiques, le stage runtime nginx:alpine sert les fichiers et fait le reverse proxy vers le backend via /api.

2.3 Orchestration Docker Compose

Le fichier docker-compose.yml définit trois services (db, backend, frontend) sur un réseau bridge dédié (appnet). La DB possède un healthcheck pg_isready et le backend déclare depends_on avec la condition service_healthy afin de ne démarrer qu'une fois la base prête.

```
services:
  db:
    image: postgres:15-alpine
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U appuser -d appdb"]
  backend:
    build: ./backend
    ports: ["5000:5000"]
    depends_on: { db: { condition: service_healthy } }
  frontend:
    build: ./frontend
    ports: ["8080:80"]
    depends_on: [backend]
```

2.4 Lancement local

```
$ cp .env.example .env
$ docker compose up --build
$ curl http://localhost:5000/health
{"service":"backend","status":"ok"}
```

3. Tests automatisés

Le backend embarque 3 tests pytest qui vérifient le comportement de l'endpoint /health, son content-type et le fait qu'une route inconnue renvoie bien un 404. Ces tests sont exécutés automatiquement au premier job du pipeline CI/CD et bloquent l'ensemble de la chaîne en cas d'échec.

```
def test_health_returns_ok(client):
    res = client.get('/health')
    assert res.status_code == 200
    assert res.get_json()['status'] == 'ok'
```

4. Machine virtuelle Azure et installation

4.1 Création de la VM

Une VM Ubuntu 22.04 (taille Standard_B2s) a été provisionnée sur Azure avec une IP publique statique 74.161.44.42. La connexion s'effectue en SSH avec authentification par mot de passe.

4.2 Installation de Docker et Docker Compose

```
$ curl -fsSL https://get.docker.com | sudo sh
$ sudo apt install -y docker-compose-plugin
$ sudo usermod -aG docker $USER && newgrp docker
```

```
azureuser@rayane-machine:~$ docker --version
Docker version 29.4.1, build 055a478
azureuser@rayane-machine:~$ docker compose version
Docker Compose version v5.1.3
```

Figure 1 — Vérification des versions de Docker et Docker Compose installés sur la VM.

4.3 Installation de Kubernetes via k3s

k3s a été retenu à la place de Minikube car il expose nativement les services de type NodePort sur l'IP de l'hôte — là où Minikube aurait nécessité un tunnel ou un reverse-proxy supplémentaire pour exposer l'application depuis l'extérieur.

```
$ curl -sfL https://get.k3s.io | sh -
$ sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config
$ sudo chown $USER:$USER ~/.kube/config
$ export KUBECONFIG=~/.kube/config
```

```
azureuser@rayane-machine:~$ kubectl get nodes
NAME             STATUS   ROLES           AGE   VERSION
rayane-machine   Ready    control-plane   49s   v1.34.6+k3s1
```

Figure 2 — Le node k3s (rayane-machine) est bien en état Ready.

```
azureuser@rayane-machine:~$ kubectl get pods -A
NAMESPACE     NAME                                      READY   STATUS      RES
ARTS   AGE
kube-system   coredns-76c974cb66-64lpj                  1/1     Running     0
       67s
kube-system   helm-install-traefik-2n8bt                0/1     Completed   1
       66s
kube-system   helm-install-traefik-crd-fgnxr            0/1     Completed   0
       66s
kube-system   local-path-provisioner-8686667995-ktrbv   1/1     Running     0
       67s
kube-system   metrics-server-c8774f4f4-hbx8j            1/1     Running     0
       67s
kube-system   svclb-traefik-6a0d2e05-p67pm              2/2     Running     0
       40s
kube-system   traefik-c5c8bf4ff-wwrvr                   1/1     Running     0
       40s
```

Figure 3 — Les pods système k3s (coredns, traefik, metrics-server...) tournent.

4.4 Ouverture du port 30080 dans le NSG Azure

Pour rendre l'application accessible depuis l'extérieur, une règle de sécurité entrante a été ajoutée au NSG de la VM pour autoriser le port TCP 30080 (celui exposé par le service Kubernetes frontend).

300	SSH		22	TCP	Tous	Tous	Allow
310	Port-30080		30080	TCP	Tous	Tous	Allow

Figure 4 — Règles NSG de la VM : SSH (port 22) et NodePort frontend (port 30080) autorisés.

5. Déploiement Kubernetes

5.1 Manifests utilisés

- namespace.yml — crée le namespace applicatif.
- secret.yml — stocke les credentials PostgreSQL (type Opaque).
- db-deployment.yml — Deployment + Service ClusterIP pour PostgreSQL.
- backend-deployment.yml — Deployment (2 replicas, RollingUpdate) avec livenessProbe et readinessProbe HTTP, resources requests/limits, et un Service ClusterIP.
- frontend-deployment.yml — Deployment Nginx et Service de type NodePort exposant le port 30080 sur l'IP de la VM.

5.2 Application des manifests

```
$ kubectl apply -f k8s/namespace.yml
$ kubectl apply -f k8s/secret.yml
$ kubectl apply -f k8s/db-deployment.yml
$ kubectl apply -f k8s/backend-deployment.yml
$ kubectl apply -f k8s/frontend-deployment.yml
```

```
azureuser@rayane-machine:~/k3s-tp$ ls -a
.    .env.example   .github     README.md   docker-compose.yml   k8s
..   .git           .gitignore  backend     frontend
```

Figure 5 — Le repo k3s-tp cloné sur la VM contient tous les fichiers du projet.

```
azureuser@rayane-machine:~/k3s-tp$ kubectl get pods -n devops-tp -w
NAME                        READY   STATUS    RESTARTS   AGE
backend-67b6cbcd86-5dkwp    1/1     Running   0          25m
backend-67b6cbcd86-fhqbd    1/1     Running   0          25m
db-59ccbccf4f-z5rdx         1/1     Running   0          25m
frontend-7f99cf6f6d-sxhrf   1/1     Running   0          25m
```

Figure 6 — Tous les pods applicatifs (db, backend x2, frontend) sont en état Running.

```
^Cazureuser@rayane-machine:~/k3s-tp$ kubectl get svc -n devops-tp
NAME       TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
backend    ClusterIP   10.43.253.6     <none>        5000/TCP       26m
db         ClusterIP   10.43.156.131   <none>        5432/TCP       26m
frontend   NodePort    10.43.130.191   <none>        80:30080/TCP   26m
```

Figure 7 — Services Kubernetes : le frontend est exposé en NodePort 80:30080/TCP.

6. Validation de l'accès externe

Depuis le poste local, l'application est accessible sur http://74.161.44.42:30080. Le frontend affiche le statut du backend, et une requête directe sur l'API confirme le bon fonctionnement du reverse proxy Nginx vers le service backend Kubernetes.

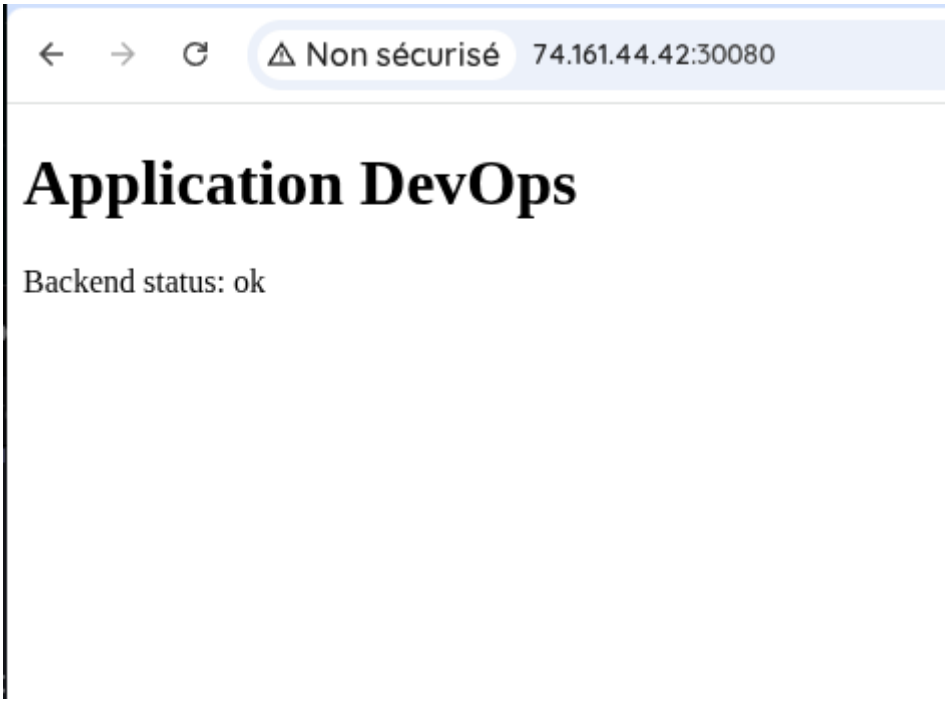

Figure 8 — Accès à l'application depuis un navigateur (IP publique de la VM, port 30080).

```
→  reference-devops-tp (main) curl http://74.161.44.42:30080/api/health
{"service":"backend","status":"ok"}
```

Figure 9 — curl sur /api/health via le reverse proxy Nginx retourne la réponse du backend.

7. Pipeline CI/CD avec GitHub Actions

7.1 Structure du workflow

Le workflow .github/workflows/ci-cd.yml est composé de 3 jobs chaînés. Un échec dans un job précédent bloque le suivant grâce à la directive needs, ce qui garantit qu'aucune image cassée n'est poussée sur le registry et qu'aucun déploiement n'est tenté si les tests échouent.

```
Job             Rôle                                              Bloquant
---             ----                                              --------
test            pytest du backend                                 Oui
build-and-push  Build multi-arch Buildx + push Docker Hub         Oui
deploy          SSH vers la VM, kubectl set image, rollout status Oui
```

7.2 Séquence d'exécution

```
git push main
     |
     v
[test] -- pytest test_app.py -v (3 tests)
     |  ok
     v
[build-and-push] -- docker buildx backend + frontend
                 -- push saphir10036/devops-tp-rayane-*:{latest,<sha>}
     |  ok
     v
[deploy] -- ssh azureuser@74.161.44.42
         -- git reset --hard origin/main
         -- kubectl apply -f k8s/
         -- kubectl set image deployment/backend ... :<sha>
         -- kubectl set image deployment/frontend ... :<sha>
         -- kubectl rollout status (timeout 120s)
```

Figure 10 — Exécution du pipeline CI/CD sur GitHub Actions : les 3 jobs passent en 1m13s après le push du commit ci: trigger first pipeline run.

8. Gestion des variables d'environnement et secrets

Aucune valeur sensible n'est écrite en dur dans le code versionné. Trois mécanismes complémentaires sont utilisés selon le contexte :

- En local : fichier .env (git-ignoré), basé sur un .env.example fourni. Les variables sont interpolées dans le docker-compose.yml.
- Dans Kubernetes : objet Secret de type Opaque (k8s/secret.yml) et référence via secretKeyRef depuis les Deployments.
- Dans GitHub Actions : secrets de repository référencés via ${{ secrets.NOM_DU_SECRET }} dans le workflow.

Secrets configurés dans GitHub Actions

```
DOCKER_USERNAME   saphir10036
DOCKER_PASSWORD   token Docker Hub (jamais le mot de passe du compte)
VM_HOST           74.161.44.42
VM_USER           azureuser
VM_PASSWORD       mot de passe SSH de la VM
```

9. Bonus réalisés

- Multi-stage build sur les deux Dockerfiles (backend et frontend).
- Liveness + readiness probes HTTP sur le backend Kubernetes.
- Rolling update sans downtime (2 replicas backend, maxUnavailable=0).
- Resource requests/limits explicites sur tous les pods.
- k3s à la place de Minikube (choix justifié pour l'exposition NodePort).
- Tag d'image par SHA de commit en plus de latest, pour garantir une traçabilité stricte.

10. Difficultés rencontrées

10.1 Exposition du NodePort avec Minikube

Problème initial : Minikube crée un réseau isolé et le NodePort n'est pas joignable directement depuis l'IP publique de la VM. Les tentatives minikube tunnel et socat ont toutes deux fonctionné mais introduisent un point de défaillance supplémentaire.

Solution retenue : remplacement de Minikube par k3s, qui expose nativement les NodePorts sur l'IP de l'hôte. L'énoncé autorisait explicitement les deux.

10.2 Ordre de démarrage DB puis backend

Problème : le backend tombait en connection refused au démarrage, PostgreSQL n'étant pas encore prêt à accepter les connexions.

Solution : en Docker Compose, depends_on.condition: service_healthy couplé à un healthcheck pg_isready. En Kubernetes, readinessProbe HTTP sur /health côté backend qui retarde l'entrée du pod dans le Service tant que la DB n'est pas atteignable.

10.3 Conflit de NodePort lors du redéploiement

Problème : lors du passage d'une première version (namespace devops-tp) au stack définitif (namespace devops-tp-rayane), l'application du manifest frontend a échoué avec « provided port is already allocated » car l'ancien Service n'avait pas encore été supprimé.

Solution : suppression explicite de l'ancien namespace avec kubectl delete namespace devops-tp avant ré-application, ce qui garbage-collecte les Services qui y étaient rattachés.

10.4 Accès non-root à kubectl sur k3s

Problème : le fichier kubeconfig de k3s (/etc/rancher/k3s/k3s.yaml) n'est lisible qu'en root par défaut, ce qui faisait échouer les commandes kubectl lancées par le job SSH du CI.

Solution : copie de k3s.yaml dans ~/.kube/config du user azureuser, chown, et export KUBECONFIG dans ~/.bashrc.

11. Conclusion

L'ensemble des objectifs du TP a été atteint : l'application full stack est conteneurisée (multi-stage builds), orchestrée localement avec Docker Compose, déployée sur un cluster Kubernetes (k3s) installé sur une VM Azure, et toute la chaîne est automatisée par un pipeline GitHub Actions à trois étapes (test, build & push, deploy). Les secrets sont gérés de bout en bout sans apparaître dans le code versionné. Le pipeline s'est exécuté avec succès en 1 minute 13 (figure 10) et l'application est accessible sur http://74.161.44.42:30080.